

GIÁO TRÌNH PYTHON CƠ BẢN

Dành cho người mới bắt đầu lập trình

Học phần 2 tín chỉ · 8 buổi · Tài liệu nội bộ

Khoa Công nghệ Giáo dục

Lời nói đầu và mục tiêu học phần

Học phần giúp người học chưa từng lập trình viết được các chương trình Python nhỏ để xử lý dữ liệu hằng ngày: tính toán, lọc danh sách, đọc ghi tệp. Sau học phần, người học có thể:

- Cài đặt Python và chạy chương trình từ tệp .py.
- Dùng biến, kiểu dữ liệu cơ bản và phép toán.
- Viết câu lệnh điều kiện và vòng lặp để xử lý danh sách.
- Tách chương trình thành các hàm có tên rõ ràng.
- Làm việc với list và dict, đọc ghi tệp văn bản và JSON.

Đánh giá: bài tập hằng tuần 40%, dự án cuối khóa 60%. Dự án được chấm theo rubric công bố cùng đề bài.

Chương 1. Làm quen với Python

Python là ngôn ngữ lập trình thông dịch, cú pháp gần với ngôn ngữ tự nhiên. Chương trình Python là một tệp văn bản có đuôi .py, được trình thông dịch đọc và thực hiện từ trên xuống dưới.

Cài đặt. Tải Python 3 từ python.org, khi cài trên Windows nhớ chọn "Add Python to PATH". Kiểm tra bằng lệnh `python --version` trong cửa sổ dòng lệnh.

Chương trình đầu tiên.

```
print("Xin chào, Python!")
```

Lưu thành `hello.py` rồi chạy `python hello.py`. Hàm `print` in giá trị ra màn hình. Mỗi câu lệnh nằm trên một dòng; Python dùng thụt lề (4 dấu cách) để nhóm câu lệnh thay cho dấu ngoặc nhọn.

Chú thích bắt đầu bằng dấu `#` và bị trình thông dịch bỏ qua. Viết chú thích để giải thích *vì sao*, không lặp lại điều code đã nói rõ.

Chương 2. Biến và kiểu dữ liệu

Biến là tên gắn với một giá trị. Gán giá trị bằng dấu=:

```
ten = "An"  
tuoi = 20  
chieu_cao = 1.68  
dang_hoc = True
```

Bốn kiểu cơ bản: int (số nguyên), float (số thực), str (chuỗi), bool (đúng/sai). Hàm type() cho biết kiểu của giá trị.

Quy tắc đặt tên: chữ thường, nối bằng dấu gạch dưới (so_tien, tong_chi), không bắt đầu bằng chữ số, không trùng từ khóa. Tên biến nên nói rõ ý nghĩa: tong_chi tốt hơn t hay x1.

Chuyển kiểu: int("12"), float("3.5"), str(10). Dữ liệu người dùng nhập bằng input() luôn là chuỗi, cần chuyển kiểu trước khi tính toán.

Chương 3. Phép toán và biểu thức

Phép toán số học: + - * /, chia lấy phần nguyên//, chia lấy dư%, lũy thừa **. Phép / luôn trả về float.

```
tong = 120000 + 45000
trung_binh = tong / 2
so_chan = 10 % 2 == 0
```

Phép so sánh: == != < > <= >= trả về bool. Phép logic: and, or, not.

f-string giúp chèn giá trị vào chuỗi:

```
print(f"Tổng chi: {tong:,.} đồng")
```

Lỗi thường gặp: cộng chuỗi với số ("Tổng: " + 5) gây TypeError; hãy dùng f-string hoặc str().

Chương 4. Câu lệnh điều kiện

Câu lệnh if cho phép chương trình chọn nhánh thực hiện:

```
if so_tien > 500000:  
    print("Khoản chi lớn")  
elif so_tien > 100000:  
    print("Khoản chi vừa")  
else:  
    print("Khoản chi nhỏ")
```

Chỉ một nhánh được chạy: nhánh đầu tiên có điều kiện đúng. Thân mỗi nhánh phải thụt lề cùng mức.

Điều kiện có thể kết hợp: `if tuoi >= 18 and co_the:`. Tránh lồng quá nhiều tầng if; khi điều kiện phức tạp, đặt nó vào một biến có tên rõ nghĩa, ví dụ `la_khoan_bat_thuong = so_tien > 5 * trung_binh`.

Chương 5. Vòng lặp for và while

Vòng lặp for duyệt lần lượt từng phần tử của một dãy:

```
khoan_chi = [120000, 45000, 300000]
tong = 0
for so_tien in khoan_chi:
    tong = tong + so_tien
print(tong)
```

range(n) sinh các số 0..n-1: for i in range(5):. Vòng lặp while chạy khi điều kiện còn đúng, dừng khi chưa biết trước số lần lặp.

break thoát vòng lặp ngay; continue bỏ qua phần còn lại của lượt hiện tại.

Mẫu tích lũy: khởi tạo biến kết quả trước vòng lặp, cập nhật trong vòng lặp, dừng sau vòng lặp. Đây là mẫu dùng để tính tổng, đếm, tìm lớn nhất. Lỗi hay gặp: khởi tạo tong = 0 bên trong vòng lặp nên kết quả bị đặt lại mỗi lượt.

Chương 6. Hàm

Hàm gom một đoạn xử lý thành khối có tên, nhận tham số và trả về kết quả bằng `return`:

```
def tinh_tong(khoan_chi):  
    tong = 0  
    for so_tien in khoan_chi:  
        tong += so_tien  
    return tong
```

Hàm tốt làm **một việc**, có tên động từ mô tả việc đó (`tinh_tong`, `loc_theo_danh_muc`), và không in ra màn hình khi nhiệm vụ của nó là tính toán — để phần gọi hàm quyết định in hay không.

Phân biệt `print` và `return`: `print` chỉ hiển thị, `return` trao giá trị cho nơi gọi để dùng tiếp. Hàm không có `return` trả về `None`.

Tham số mặc định: `def dinh_dang(so_tien, don_vi="đồng"):`.

Chương 7. Danh sách (list)

List chứa nhiều giá trị theo thứ tự: `ds = [3, 1, 2]`. Truy cập theo chỉ số từ 0: `ds[0]`; chỉ số âm lấy từ cuối: `ds[-1]`.

Thao tác thường dùng: `append` thêm cuối, `len` đếm phần tử, `sorted` trả về danh sách đã sắp xếp, `sum`, `max`, `min`.

List comprehension tạo danh sách mới từ danh sách cũ:

```
lon = [x for x in khoan_chi if x > 100000]
```

Cắt lát: `ds[1:3]` lấy phần tử 1 và 2. Lỗi `IndexError` xảy ra khi truy cập chỉ số không tồn tại.

Chương 8. Từ điển (dict)

Dict lưu cặp khóa-giá trị, tra cứu theo khóa:

```
khoan = {"so_tien": 120000, "danh_muc": "an uong", "ngay": "2026-03-02"}  
print(khoan["danh_muc"])
```

Dùng get khi khóa có thể không có: `khoan.get("ghi_chu", "")`. Duyệt dict bằng for khoa, gia_tri in `khoan.items()`:

Gom nhóm là mẫu quan trọng: cộng dồn số tiền theo danh mục.

```
theo_danh_muc = {}  
for k in cac_khoan:  
    dm = k["danh_muc"]  
    theo_danh_muc[dm] = theo_danh_muc.get(dm, 0) + k["so_tien"]
```

Chương 9. Làm việc với tệp

Đọc tệp văn bản với `with open(...)` để tệp được đóng tự động:

```
with open("chi_tieu.txt", encoding="utf-8") as f:
    for dong in f:
        print(dong.strip())
```

Luôn ghi rõ `encoding="utf-8"` khi làm việc với tiếng Việt. Chế độ `"w"` ghi đè, `"a"` ghi thêm vào cuối.

JSON là định dạng phổ biến để lưu list/dict:

```
import json
with open("chi_tieu.json", encoding="utf-8") as f:
    cac_khoan = json.load(f)
```

`json.dump(du_lieu, f, ensure_ascii=False, indent=2)` ghi ra tệp, giữ nguyên dấu tiếng Việt.

Chương 10. Tổ chức chương trình và kiểm thử đơn giản

Chương trình lớn hơn nên chia thành nhiều hàm nhỏ và một hàm `main()` gọi chúng theo thứ tự. Đặt lời gọi trong khối:

```
if __name__ == "__main__":  
    main()
```

để tệp có thể được import mà không tự chạy.

Kiểm thử bằng `assert`: viết vài dòng kiểm tra kết quả mong đợi của hàm, ví dụ `assert tinh_tong([1, 2]) == 3`. Với nhiều trường hợp, dùng thư viện `pytest`: đặt các hàm `test_...` trong tệp `test_*.py` và chạy lệnh `pytest`.

Nên kiểm tra cả trường hợp biên: danh sách rỗng, một phân tử, số âm.

Phụ lục A. Bảng lỗi thường gặp

- **IndentationError**: thụt lề không đều; dùng 4 dấu cách, không trộn Tab.
- **NameError**: dùng biến chưa gán hoặc gõ sai tên.
- **TypeError**: phép toán giữa hai kiểu không hợp (chuỗi + số).
- **KeyError**: truy cập khóa không có trong dict; dùng get.
- **FileNotFoundError**: sai đường dẫn tệp; kiểm tra thư mục đang chạy.

Khi gặp lỗi, đọc dòng cuối của thông báo (loại lỗi) và số dòng được chỉ ra trước khi sửa.

Phụ lục B. Lịch học 8 buổi

1. Chương 1-2: cài đặt, biến, kiểu dữ liệu.
2. Chương 3-4: biểu thức, điều kiện.
3. Chương 5: vòng lặp.
4. Chương 6: hàm.
5. Chương 7-8: list, dict.
6. Chương 9: tệp và JSON.
7. Chương 10: tổ chức chương trình, kiểm thử.
8. Bảo vệ dự án cuối khóa.

